QueryAPIGate DOCUMENTATION · REFERENCE

Example APIs

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=EXAMPLES

Four worked scenarios you can load, try and remove again - each a collection of saved queries plus a role shaped like the key that would front it - plus a fifth role that spans two of those collections at once. They show what QueryAPIGate is *for*: a named, small, rate-limited surface over a database, instead of a hand-rolled controller.

```shell
pip install queryapigate
queryapigate examples load          # installs everything below, and prints a real API key per scenario
-
                                     # store them now, they cannot be shown again (see below)
queryapigate serve                  # then open http://127.0.0.1:5000/console - a key from above is now
required
queryapigate examples unload        # removes exactly what `load` installed - nothing else
```

The data is a small SQLite database generated on your machine (`examples.db`, about 3 MB, 8 tables: 60 films, 200 customers, 20,000 rentals over the last 120 days (dated relative to the day you load them so the "today" and "overdue" numbers always have something to show), plus `category`, `store`, `staff`, `address` and `payment` - every rental has a real payment record, processed by a real staff member at a real store, so the schema has real foreign keys to explore, not just `film` / `customer` / `rental`. Nothing third-party is shipped. The connection is called `examples`.

Loading, hiding and removing them

`queryapigate examples load`	Install them. Idempotent: running it again changes nothing (no extra versions), and it finishes an interrupted load.
`queryapigate examples unload`	Remove exactly what was installed - the queries, roles, keys, connection and database file.
`queryapigate examples status`	Say whether they are loaded (`Loaded`, `Partly loaded`, or `Not loaded`).
`QUERYAPIGATE_LOAD_EXAMPLES=yes`	Load them at server start - for a container (`docker run -e QUERYAPIGATE_LOAD_EXAMPLES=yes -v qag:/data ...`). Idempotent across restarts. Unset, or `no`, leaves things as they are: it never removes anything - use `unload`.
Admin UI	Once loaded, API Repository shows an "Example APIs are loaded" bar with **Remove examples** (and **Finish loading** after an interrupted load). Loading itself is the command above, the variable below or the API.
`POST` / `DELETE` / `GET` `/api/v1/examples`	The same, over HTTP (admin only).

Everything installed is **marked** `example` (`"example": true` on each query, role, key and connection), and removal deletes exactly what is marked:

- A query, role, key, connection or file of yours that merely shares a name is **never overwritten and never removed** - loading stops with a clear message and changes nothing until you rename it.
- If you edit the `examples` connection in the admin UI it stops being marked (it is yours now), so `unload` leaves it and a later `load` reports the conflict.
- **A real API key is created for every role** (`role: "<name>"`, the same "create key from role" the admin UI itself uses) - the server now requires a key for *every* request, not just the example ones, the moment these exist. Each key's secret is shown exactly once: printed by `queryapigate examples load`, returned in `POST /api/v1/examples`'s response, or logged a single time at server startup if you used `QUERYAPIGATE_LOAD_EXAMPLES` instead - store it then, it is never shown again (the same rule every API key already has). Want an *additional*, differently-scoped or differently-named key for a specific team instead of the auto-created one? The `curl` examples under each scenario below still show how, from the same role.
- Each freshly-loaded query also gets a handful of realistic-looking, real `execution_history` entries, so the admin UI's History tab, Home tab and requests-per-day chart show something immediately - a second `load` never adds more. The API Keys/Connections "Usage" columns and the Metrics tab are **not** seeded this way: those reflect real live traffic only, reset on every restart, and stay at zero until you actually call something.
- If you granted a key one of the example collections, `unload` tells you, and that grant now reaches nothing (see `GET /api/v1/collections`).

1. Reporting API - `examples-reporting`

A read-only, rate-limited key scoped to a few aggregate queries. Role `example-reporting`: no connections, the `examples-reporting` collection, read-only, `200/hour`.

QUERY	WHAT IT DOES	PARAMETERS
`example_monthly_revenue`	Rentals and revenue per month	`months` 1-24, default 6
`example_top_films`	Most-rented films, optionally in one category	`top_n` 1-50 (default 10), `category` (optional, one of six)
`example_revenue_by_category`	Revenue per category	`days` 1-120, default 30

```shell
curl 'http://127.0.0.1:5000/q/example_monthly_revenue?months=3'
curl 'http://127.0.0.1:5000/q/example_top_films?top_n=5&category=Comedy'
curl 'http://127.0.0.1:5000/q/example_top_films?category=Nope'      # 400: category must be one of ...
```

Give a team its own key (with `QUERYAPIGATE_API_KEY` set, using the admin key):

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: <admin key>' -H 'Content-Type: application/json' \
     -d '{"name": "finance-team", "role": "example-reporting"}'          # the secret is shown once
curl 'http://127.0.0.1:5000/q/example_monthly_revenue' -H 'X-API-Key: <the secret>'
```

That key can run these three queries and nothing else - no ad-hoc SQL, no other queries, and no other connection.

2. Dashboard data API - `examples-dashboard`

Small queries meant to be polled every few seconds. Each is cached for **30 seconds** (`cache_ttl`), so a wall of dashboards polling costs one database query per half minute, not one per poll. Role `example-dashboard`, `600/minute`.

QUERY	WHAT IT DOES
`example_kpi_rentals_today`	Rentals and revenue so far today (UTC)
`example_kpi_active_rentals`	Rentals currently out
`example_kpi_overdue`	Out for more than 7 days
`example_recent_rentals`	A live feed (`hours` 1-168, default 24)

```shell
curl -i 'http://127.0.0.1:5000/q/example_kpi_active_rentals'    # X-Cache: MISS
curl -i 'http://127.0.0.1:5000/q/example_kpi_active_rentals'    # X-Cache: HIT, with an ETag
```

3. Data export API - `examples-export`

A ~20,000-row export meant for streaming exports: constant memory on the server however large the result. `example_all_rentals` joins six tables - `rental`, `film`, `customer`, `payment`, `staff` and `store` - so it doubles as a real, joined dataset to try the Access tab's Query flow diagram on. Role `example-export`, `20/hour`.

```shell
curl 'http://127.0.0.1:5000/q/example_all_rentals?stream=true&format=csv' -o rentals.csv
curl 'http://127.0.0.1:5000/q/example_rentals_since?since=2026-09-01&stream=true&format=ndjson'   #
incremental
```

Or write it to a file from cron with no server running, using the CLI export: `queryapigate export example_all_rentals --out '/exports/{name}_{date}.csv'`.

4. Partner / integration API - `examples-partner`

What you would hand an external company: a narrow, parameter-validated surface that cannot reach anything else. Role `example-partner`: only the `examples-partner` collection, read-only, `60/minute`.

QUERY	WHAT IT DOES	RULES
`example_film_lookup`	One film and how it is doing	`film_id` required, integer >= 1
`example_film_search`	Find films by part of a title	`text` required, 2-30 letters and spaces

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: <admin key>' -H 'Content-Type: application/json' \
     -d '{"name": "acme-corp", "role": "example-partner", "expires_at": "2026-12-31"}'   # expiry is per key
curl 'http://127.0.0.1:5000/q/example_film_lookup?film_id=7'                -H 'X-API-Key: <acme secret>'   # 200
curl 'http://127.0.0.1:5000/q/example_top_films'                            -H 'X-API-Key: <acme secret>'   # 403
curl 'http://127.0.0.1:5000/q/example_film_lookup?film_id=7&connection_name=other' -H 'X-API-Key: <acme secret>'   # 403
curl -X POST http://127.0.0.1:5000/execute_sql -H 'X-API-Key: <acme secret>' -H 'Content-Type: application/json' \
     -d '{"sql": "SELECT * FROM rental", "connection_name": "examples"}'
# 403
```

Add a query to the `examples-partner` collection later and this key can run it with no change to the key - that is what a collection grant is for. Export the whole set to Postman from the admin UI's **Postman** button on the collection.

5. A role spanning two collections - `example-executive`

None of the four scenarios above shows this on its own: a role - and so a key created from it - can be granted more than one collection at once. `example-executive` is read-only, `300/hour`, and reaches every query in **both** `examples-reporting` and `examples-dashboard` - nothing from `examples-export` or `examples-partner`.

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: <admin key>' -H 'Content-Type: application/json' \
     -d '{"name": "exec-dashboard", "role": "example-executive"}'
curl 'http://127.0.0.1:5000/q/example_monthly_revenue'       -H 'X-API-Key: <exec secret>'   # 200 - reporting
curl 'http://127.0.0.1:5000/q/example_kpi_active_rentals'    -H 'X-API-Key: <exec secret>'   # 200 - dashboard
curl 'http://127.0.0.1:5000/q/example_all_rentals'           -H 'X-API-Key: <exec secret>'   # 403 - export
```

Granting a second collection to any role - yours or this one - works the same way: add its name to the role's `collections` list and every key created from that role gains it immediately.

Things to try next

- **Admin UI** (`/console`): the API Repository groups these by collection; **Move…** shows which keys would gain or lose access before anything changes; the Audit Log records every change.
- **Docs and OpenAPI**: `/docs` lists exactly the queries the key you paste in can run - try it with the partner key.
- **Schema browser** (API Designer → Schema → Columns): PK/FK badges throughout - `rental.film_id` / `customer_id`, `payment.rental_id` / `staff_id`, `staff.store_id` and `address.customer_id` are all real foreign keys, not just similarly-named columns. `category` has a primary key and no foreign key into it from anywhere - a deliberate contrast, a plain lookup table rather than something joined into the example queries.

- **Query flow diagram** (API Repository → `example_all_rentals` → Access tab): a real node-link diagram of all six tables and five joins that query's SQL actually touches (`rental` → `film` / `customer` / `payment` → `staff` → `store`), built from real SQL parsing, not a text-search guess.
- **Table-scoped autocomplete**: in the API Designer against the `examples` connection, type `r.` or `f.` after `FROM rental r JOIN film f ON f.film_id = r.film_id` - a popup of that table's real columns.
- **Real SQL pretty-printing**: the SQL tab of any of these queries shows a real, `sqlglot`-formatted rendering, not just a plain reflow.
- **Your own**: copy one of these queries into a new saved query pointed at your own connection - the SQL, parameter rules and `cache_ttl` carry over unchanged.

READ NEXT
Query metadata

QUESTIONS OR PROBLEMS
Discord · GitHub issues

LATEST VERSION OF THIS PAGE
anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=EXAMPLES

FOLLOW QUERYAPIGATE GitHub X Discord Bluesky Mastodon YouTube Reddit

@queryapigate on X, Bluesky, Mastodon and YouTube · r/queryapigate on Reddit · github.com/AnanthaRajuC/QueryAPIGate

QueryAPIGate · one governed gate between your databases and everything that asks them for data · self-hosted, Apache-2.0 · © 2026 QueryAPIGate